

**UTN Facultad Regional
Mendoza**

**Tecnicatura
Universitaria en
Programación**

Trabajo Práctico Integrador

Base de Datos FOOD STORE

Grupo: Xavier Pappalardo, Lautaro Gómez,
Nicolás Ibáñez, Maximiliano Reinoso,
Benjamín Arrollo y Martín Sisterna

13/08/26

LINK REPOSITORIO:

https://github.com/XavierPappalardo/UTN-TUPaD-TPI_Base_de_Datos_Food_Store_2PRO1

Índice General:

Acá tenés el Índice General completamente actualizado y ordenado para reflejar con precisión la nueva estructura de la Sección 3 (con el DER, el análisis de participaciones, las preguntas guía y el diccionario de atributos):

Índice General

1. Introducción y Objetivos
2. Marco Teórico
 - 2.1. Modelo Relacional e Integridad
 - 2.2. Normalización (1FN a BCNF)
 - 2.3. Transacciones y Propiedades ACID
3. Modelado del Dominio
 - 3.1. Diagrama Entidad-Relación (DER) Conceptual, Participaciones y Preguntas Guía
 - 3.2. Diccionario de Entidades y Atributos
 - 3.3. Esquema Relacional Definitivo y Justificación de Atributos Comunes
4. Demostración de Normalización
 - 4.1. Dependencias Funcionales
 - 4.2. Formas Normales
 - 4.3. Desnormalización Controlada: **total_pedido**
5. Implementación Física (**schema.sql**)
6. Lógica de Servidor (**objects.sql**)
 - 6.1. Vistas, Funciones y Triggers
 - 6.2. Procedimiento Almacenado **sp_crear_pedido**
7. Optimización con Índices y **EXPLAIN ANALYZE**
8. Pruebas de Transacciones y Concurrencia (**transacciones.sql**)
 - 8.1. Atomicidad y Rollback
 - 8.2. Control de Sobreventa con **FOR UPDATE**
9. Resolución de Historias de Usuario (**queries.sql**)
10. Dificultades y Soluciones
11. Conclusiones y Bibliografía

1. Introducción y Objetivos

El objetivo de este proyecto es diseñar e implementar la base de datos para **Food Store** (un negocio de comidas) en **PostgreSQL 16+**.

A diferencia de *Programación 2* —donde la base solo guardaba lo que enviábamos desde Java—, acá **la base de datos es el centro del sistema**. Nos enfocamos en diseñar bien las tablas, evitar datos duplicados y hacer que las reglas del negocio (precios, stock, totales) se controlen directamente en el motor usando SQL.

Objetivos principales:

- **Diseñar las tablas:** Crear el diagrama y relacionar las entidades con sus claves correspondientes.
- **Evitar datos repetidos (Normalizar):** Organizar la estructura para no tener redundancias ni fallos al modificar datos.
- **Poner validaciones:** Asegurar que no ingresen datos inválidos (como precios negativos o mails duplicados).
- **Programar en la base:** Usar vistas, triggers que calculen totales automáticos y un procedimiento para registrar pedidos.
- **Manejar compras concurrentes:** Evitar la sobreventa de stock y asegurar que si un pedido falla, se cancele completo.
- **Mejorar el rendimiento:** Crear índices para acelerar las búsquedas más frecuentes.

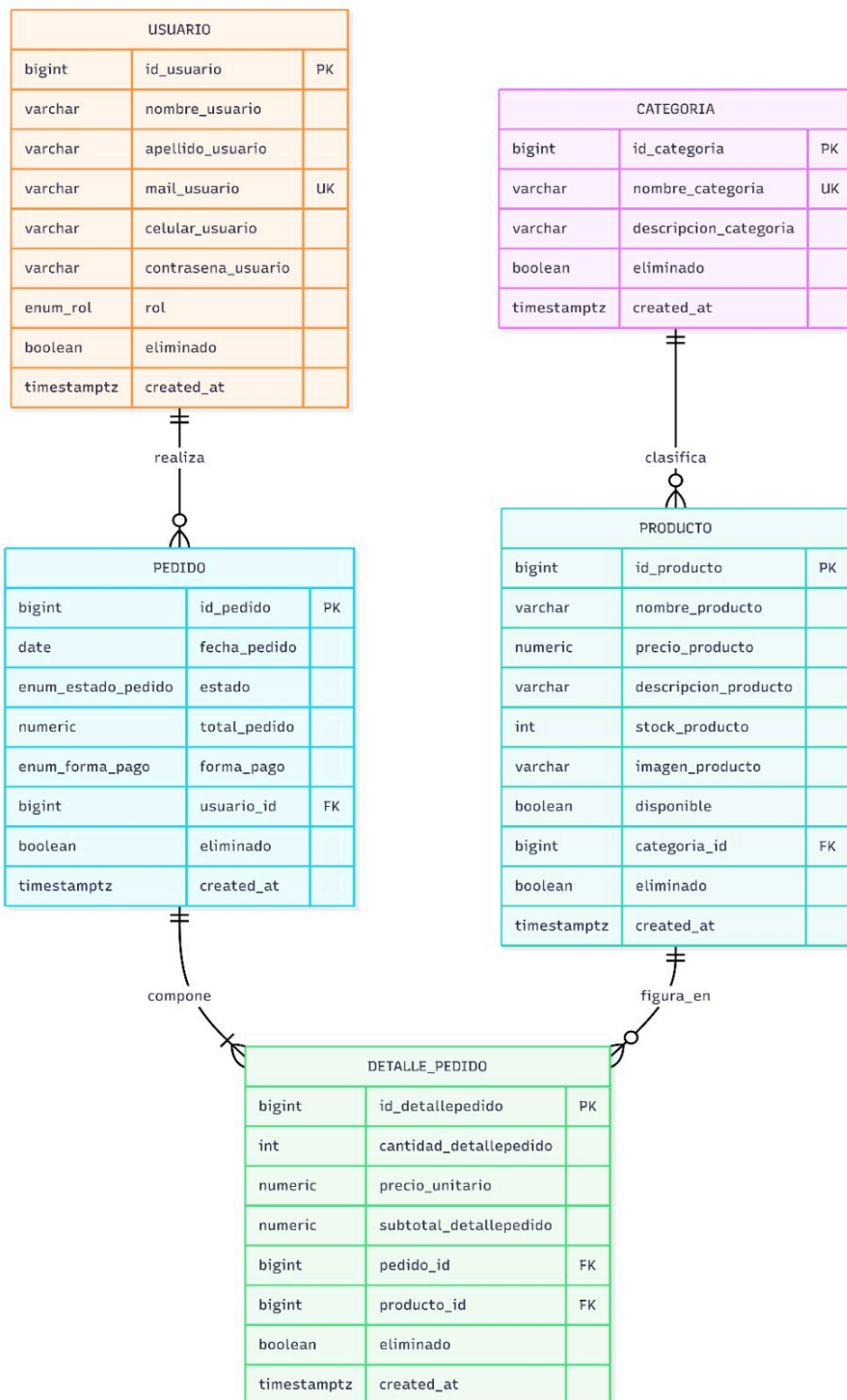
2. Marco Teórico

- **2.1. Modelo Relacional e Integridad:** Nos basamos en tablas relacionales con integridad de entidad (**PRIMARY KEY**), referencial (**FOREIGN KEY**) y de dominio (**NOT NULL, CHECK, ENUM**).
- **2.2. Normalización:** Aplicamos el proceso formal para eliminar redundancias mediante 1FN, 2FN, 3FN y BCNF.
- **2.3. Transacciones y ACID:** Garantizamos **Atomicidad** (todo o nada), **Consistencia** (cumplimiento de reglas), **Aislamiento** (sin interferencias concurrentes) y **Durabilidad** (cambios permanentes tras **COMMIT**).

3. Modelado del Dominio

3.1. Esquema Relacional Definitivo

Alineado exactamente al diagrama de la cátedra con prefijos por entidad:



Análisis de Cardinalidades y Participación

- **CATEGORIA (1) — (N) PRODUCTO (Relación: Clasifica)**
 - **Cardinalidad:** 1:N. Una categoría agrupa varios productos, pero un producto pertenece a una sola categoría.
 - **Participación en CATEGORIA:** Parcial (Opcional). Puede existir una categoría creada en el sistema que todavía no tenga productos cargados.

- **Participación en PRODUCTO:** Total (Obligatoria). Todo producto cargado debe pertenecer sí o sí a una categoría activa del menú.
- **USUARIO (1) — (N) PEDIDO (Relación: Realiza)**
 - **Cardinalidad:** 1:N. Un usuario puede hacer muchos pedidos en el tiempo. Un pedido pertenece a un solo usuario.
 - **Participación en USUARIO:** Parcial (Opcional). Un usuario recién registrado en la app puede no haber realizado ningún pedido todavía.
 - **Participación en PEDIDO:** Total (Obligatoria). Todo pedido registrado debe estar asociado obligatoriamente a un usuario para saber a quién cobrarle.
- **PEDIDO (1) — (N) DETALLE_PEDIDO (Relación: Compone)**
 - **Cardinalidad:** 1:N. Un pedido se compone de uno o varios ítems comprados. Un ítem de detalle pertenece únicamente a un pedido.
 - **Participación en PEDIDO:** Total (Obligatoria). Un pedido no puede existir sin al menos un producto comprado en su detalle.
 - **Participación en DETALLE_PEDIDO:** Total (Obligatoria). Un renglón de detalle no puede quedar colgado sin estar asociado a un pedido.
- **PRODUCTO (1) — (N) DETALLE_PEDIDO (Relación: Figura en)**
 - **Cardinalidad:** 1:N. Un producto puede aparecer vendido en muchos detalles de diferentes pedidos. Un renglón de detalle refiere a un solo producto.
 - **Participación en PRODUCTO:** Parcial (Opcional). Un producto de la carta puede no haber sido comprado nunca por ningún cliente.
 - **Participación en DETALLE_PEDIDO:** Total (Obligatoria). Cada renglón del detalle debe hacer referencia obligatoria a un producto existente.

Respuestas a las Preguntas Guía

- **¿Por qué la relación entre PRODUCTO y PEDIDO no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?**
 - **Respuesta:** No puede ser 1:N porque un pedido puede llevar varios productos y un producto se puede vender en muchos pedidos (relación N:M, muchos a muchos). Si fuera 1:N simple, solo podríamos guardar un solo producto por cada pedido realizado.
 - **Información que se perdería:** Perderíamos la posibilidad de comprar varios productos en un mismo pedido, la cantidad individual comprada de cada ítem, el subtotal por renglón y el precio congelado al momento del alta (necesario para que los pedidos viejos no cambien de precio cuando aumenta la carta).
- **¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?**
 - **Respuesta:** CATEGORIA tiene participación parcial (opcional) y PRODUCTO tiene participación total (obligatoria).
 - **Si se invirtiera la lectura:** Significaría que no podríamos crear una categoría nueva si no le agregamos un producto en ese mismo instante. Y al mismo tiempo, permitiría crear productos "huérfanos" sin ninguna categoría asignada, lo que rompería la regla del menú organizado.
- **¿Alguno de tus atributos podría descomponerse en partes más simples? Justificá tu decisión de dejarlo compuesto o de separarlo.**

- **Respuesta:** Sí, en la entidad USUARIO se dividió el nombre completo en nombre_usuario y apellido_usuario.
- **Justificación:** Esto nos permite ordenar y buscar fácilmente a los usuarios por su apellido en las listas del sistema. En cambio, atributos como descripcion_producto se dejaron en un solo texto simple porque son descripciones generales que no necesitan buscarse por partes.

3.2. Diccionario de Entidades y Atributos

Entidad: CATEGORIA

- id_categoria: Numérico (Entero) | Clave Primaria (PK) | Identificador único de la categoría.
- nombre_categoria: Texto | Único (UNIQUE) | Nombre de la categoría (ej: "Pizzas").
- descripcion_categoria: Texto | Opcional | Detalle opcional de la categoría.
- eliminado: Booleano | Requerido | Marca de borrado lógico (soft delete).
- created_at: Fecha / Hora | Requerido | Fecha de creación.

Entidad: PRODUCTO

- id_producto: Numérico (Entero) | Clave Primaria (PK) | Identificador único del producto.
- nombre_producto: Texto | Requerido | Nombre comercial del producto.
- precio_producto: Numérico (Decimal) | Requerido (≥ 0) | Precio actual en la carta.
- descripcion_producto: Texto | Opcional | Descripción e ingredientes.
- stock_producto: Numérico (Entero) | Requerido (≥ 0) | Cantidad disponible en stock.
- imagen_producto: Texto | Opcional | URL o ruta de la foto.
- disponible: Booleano | Requerido | Indica si está activo para venta.
- categoria_id: Numérico (Entero) | Clave Foránea (FK) | Relación obligatoria con CATEGORIA.
- eliminado: Booleano | Requerido | Marca de borrado lógico.
- created_at: Fecha / Hora | Requerido | Fecha de alta del producto.

Entidad: USUARIO

- id_usuario: Numérico (Entero) | Clave Primaria (PK) | Identificador único del usuario.
- nombre_usuario: Texto | Requerido | Nombre del cliente.
- apellido_usuario: Texto | Requerido | Apellido del cliente.
- mail_usuario: Texto | Único (UNIQUE) | Correo electrónico del cliente.
- celular_usuario: Texto | Opcional | Teléfono de contacto.
- contrasena_usuario: Texto | Requerido | Contraseña.
- rol: Enumerado (Texto) | Requerido | Rol (ADMIN o USUARIO).
- eliminado: Booleano | Requerido | Marca de borrado lógico.
- created_at: Fecha / Hora | Requerido | Fecha de registro.

Entidad: PEDIDO

- id_pedido: Numérico (Entero) | Clave Primaria (PK) | Identificador único del pedido.
- fecha_pedido: Fecha | Requerido | Fecha de emisión.

- estado_pedido: Enumerado (Texto) | Requerido | Estado (PENDIENTE, CONFIRMADO, etc.).
- total_pedido: Numérico (Decimal) | Requerido (≥ 0) | Monto total gastado.
- forma_pago: Enumerado (Texto) | Requerido | Pago (TARJETA, EFECTIVO, etc.).
- usuario_id: Numérico (Entero) | Clave Foránea (FK) | Relación obligatoria con USUARIO.
- eliminado: Booleano | Requerido | Marca de borrado lógico.
- created_at: Fecha / Hora | Requerido | Fecha/hora del pedido.

Entidad: DETALLE_PEDIDO

- id_detallepedido: Numérico (Entero) | Clave Primaria (PK) | Identificador de la línea comprada.
- cantidad_detallepedido: Numérico (Entero) | Requerido (> 0) | Cantidad de unidades llevadas.
- precio_unitario: Numérico (Decimal) | Requerido (≥ 0) | Precio congelado al momento de comprar.
- subtotal_detallepedido: Numérico (Decimal) | Requerido (≥ 0) | Subtotal (cantidad * precio_unitario).
- pedido_id: Numérico (Entero) | Clave Foránea (FK) / Único | Relación obligatoria con PEDIDO.
- producto_id: Numérico (Entero) | Clave Foránea (FK) / Único | Relación obligatoria con PRODUCTO.
- eliminado: Booleano | Requerido | Marca de borrado lógico.
- created_at: Fecha / Hora | Requerido | Fecha/hora del registro.

3.3. Esquema Relacional Definitivo y Atributos Comunes

- **CATEGORIA** (id_categoria, nombre_categoria, descripcion_categoria, eliminado, created_at)
- **PRODUCTO** (id_producto, nombre_producto, precio_producto, descripcion_producto, stock_producto, imagen_producto, disponible, categoria_id -> CATEGORIA, eliminado, created_at)
- **USUARIO** (id_usuario, nombre_usuario, apellido_usuario, mail_usuario, celular_usuario, contrasena_usuario, rol, eliminado, created_at)
- **PEDIDO** (id_pedido, fecha_pedido, estado_pedido, total_pedido, forma_pago, usuario_id -> USUARIO, eliminado, created_at)
- **DETALLE_PEDIDO** (id_detallepedido, cantidad_detallepedido, precio_unitario, subtotal_detallepedido, pedido_id -> PEDIDO, producto_id -> PRODUCTO, eliminado, created_at)

Justificación de Atributos Comunes

En lugar de usar herencia (INHERITS) o una tabla base centralizada —que generan problemas con FKs o costos altos de JOIN—, repetimos los atributos eliminado y created_at en cada tabla. Esto optimiza el rendimiento y mantiene la integridad referencial limpia.

4. Demostración de Normalización

4.1. Dependencias Funcionales (DF)

- **CATEGORIA:** `id_categoria` -> `nombre_categoria`, `descripcion_categoria`
- **PRODUCTO:** `id_producto` -> `nombre_producto`, `precio_producto`, `stock_producto`, `categoria_id`
- **USUARIO:** `id_usuario` -> `nombre_usuario`, `apellido_usuario`, `mail_usuario`, `rol` | `mail_usuario` -> resto de campos.
- **PEDIDO:** `id_pedido` -> `fecha_pedido`, `estado_pedido`, `total_pedido`, `usuario_id`
- **DETALLE_PEDIDO:** `id_detallepedido` -> `cantidad_detallepedido`, `precio_unitario`, `subtotal_detallepedido`, `pedido_id`, `producto_id` | {`pedido_id`, `producto_id`} -> resto de campos.

4.2. Formas Normales

- **1FN:** Todos los atributos son atómicos (los detalles no se guardan como arrays dentro de `PEDIDO`).
- **2FN:** Todas las PKs son simples (`id_...`), evitando dependencias parciales.
- **3FN / BCNF:** No hay dependencias transitivas y todo determinante es una superclave (`id_...` o `mail_usuario`).

4.3. Desnormalización Controlada: `total_pedido`

Aunque `subtotal_detallepedido` y `total_pedido` son calculables, los materializamos en la base:

1. **Rendimiento:** Evita hacer un `SUM` sobre millones de filas cada vez que se lista un pedido.
2. **Consistencia:** Se recalculan automáticamente mediante triggers en la base.
3. **Congelamiento de Precio:** `precio_unitario` guarda el valor al momento de la compra, protegiendo el historial ante futuros aumentos en `PRODUCTO`.

5. Implementación Física (`schema.sql`)

```
-- Schema.sql
-- TPI Base de Datos - Food Store
--
-- Tipos enumerados
--
CREATE TYPE rol AS ENUM ('ADMIN','USUARIO');
CREATE TYPE estado_pedido AS ENUM ('PENDIENTE','CONFIRMADO',
                                     'TERMINADO','CANCELADO');
CREATE TYPE forma_pago AS ENUM ('TARJETA','TRANSFERENCIA','EFECTIVO');
--
-- Tabla: categoria
--
CREATE TABLE categoria (
  id_categoria BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre_categoria VARCHAR(80) NOT NULL UNIQUE,
  descripcion_categoria VARCHAR(255),
```

```

    eliminado          BOOLEAN    NOT NULL DEFAULT FALSE,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Tabla: producto
--
CREATE TABLE producto (
    id_producto        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_producto    VARCHAR(120) NOT NULL,
    precio_producto     NUMERIC(10,2) NOT NULL CHECK (precio_producto >= 0),
    descripcion_producto VARCHAR(255),
    stock_producto      INTEGER    NOT NULL DEFAULT 0 CHECK (stock_producto >= 0),
    imagen_producto     VARCHAR(255),
    disponible          BOOLEAN    NOT NULL DEFAULT TRUE,
    categoria_id        BIGINT     NOT NULL REFERENCES categoria(id_categoria),
    eliminado           BOOLEAN    NOT NULL DEFAULT FALSE,
    created_at          TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Tabla: usuario
--
CREATE TABLE usuario (
    id_usuario         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nombre_usuario     VARCHAR(80) NOT NULL,
    apellido_usuario   VARCHAR(80) NOT NULL,
    mail_usuario       VARCHAR(120) NOT NULL UNIQUE,
    celular_usuario    VARCHAR(30),
    contrasena_usuario VARCHAR(255) NOT NULL,
    rol                rol        NOT NULL DEFAULT 'USUARIO',
    eliminado          BOOLEAN    NOT NULL DEFAULT FALSE,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Tabla: pedido
--
CREATE TABLE pedido (
    id_pedido          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    fecha_pedido       DATE       NOT NULL DEFAULT CURRENT_DATE,
    estado_pedido      estado_pedido NOT NULL DEFAULT 'PENDIENTE',
    total_pedido       NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (total_pedido >= 0),
    forma_pago         forma_pago  NOT NULL,
    usuario_id         BIGINT     NOT NULL REFERENCES usuario(id_usuario),
    eliminado          BOOLEAN    NOT NULL DEFAULT FALSE,
    created_at         TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Tabla: detalle_pedido
--
CREATE TABLE detalle_pedido (
    id_detallepedido   BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cantidad_detallepedido INTEGER    NOT NULL CHECK (cantidad_detallepedido > 0),
    precio_unitario     NUMERIC(10,2) NOT NULL CHECK (precio_unitario >= 0),

```

```

subtotal_detallepedido NUMERIC(12,2) NOT NULL CHECK (subtotal_detallepedido >= 0),
pedido_id              BIGINT      NOT NULL REFERENCES pedido(id_pedido)
                        ON DELETE RESTRICT,
producto_id           BIGINT      NOT NULL REFERENCES producto(id_producto),
eliminado             BOOLEAN      NOT NULL DEFAULT FALSE,
created_at            TIMESTAMPTZ  NOT NULL DEFAULT now(),
UNIQUE (pedido_id, producto_id)
);
--
-- Índices requeridos
--
-- Soporta el listado de productos por categoría (FK producto -> categoría)
CREATE INDEX idx_producto_categoria_id ON producto(categoria_id);
-- Soporta el historial de pedidos por usuario (FK pedido -> usuario)
CREATE INDEX idx_pedido_usuario_id ON pedido(usuario_id);
-- Índice parcial: acelera búsquedas por nombre de producto solo entre
-- las filas vigentes (no eliminadas), que son las que consultan las vistas
CREATE INDEX idx_producto_nombre_vigente
ON producto(nombre_producto) WHERE eliminado = FALSE;

```

6. Lógica de Servidor (**objects.sql**)

6.1. Vistas, Funciones y Triggers

Implementamos triggers de nivel de sentencia (**FOR EACH STATEMENT**) con tablas de transición (**REFERENCING NEW/OLD TABLE**) para recalcular **total_pedido** sin caer en errores de mutación.

```

-- objects.sql
-- TPI Base de Datos - Food Store
--
-- Vistas obligatorias
--
CREATE VIEW v_categorias_vigentes AS
SELECT id_categoria      AS id,
       nombre_categoria  AS nombre,
       descripcion_categoria AS descripcion
FROM   categoria
WHERE  eliminado = FALSE;
CREATE VIEW v_productos_vigentes AS
SELECT p.id_producto    AS id,
       p.nombre_producto AS nombre,
       p.precio_producto AS precio,
       p.stock_producto  AS stock,
       c.nombre_categoria AS categoria
FROM   producto p
JOIN   categoria c ON c.id_categoria = p.categoria_id
WHERE  p.eliminado = FALSE AND c.eliminado = FALSE;
CREATE VIEW v_pedidos_resumen AS
SELECT ped.id_pedido AS id,
       u.nombre_usuario || ' ' || u.apellido_usuario AS usuario,
       ped.fecha_pedido AS fecha,

```

```

    ped.estado_pedido AS estado,
    ped.forma_pago,
    ped.total_pedido AS total
FROM pedido ped
JOIN usuario u ON u.id_usuario = ped.usuario_id
WHERE ped.eliminado = FALSE;
CREATE VIEW v_pedido_detalle AS
SELECT dp.pedido_id,
       pr.nombre_producto AS producto,
       dp.cantidad_detallepedido AS cantidad,
       dp.precio_unitario,
       dp.subtotal_detallepedido AS subtotal
FROM detalle_pedido dp
JOIN producto pr ON pr.id_producto = dp.producto_id
WHERE dp.eliminado = FALSE;

--
-- Función de cálculo de total (analogía con Calculable)
--
CREATE OR REPLACE FUNCTION calcular_total_pedido(p_pedido_id BIGINT)
RETURNS NUMERIC(12,2) AS $$
    SELECT COALESCE(SUM(subtotal_detallepedido), 0)
    FROM detalle_pedido
    WHERE pedido_id = p_pedido_id AND eliminado = FALSE;
$$ LANGUAGE sql STABLE;

--
-- Trigger: subtotal automático
--
CREATE OR REPLACE FUNCTION fn_set_subtotal()
RETURNS TRIGGER AS $$
BEGIN
    -- Si no se pasó precio_unitario, se congela el precio actual del producto
    IF NEW.precio_unitario IS NULL THEN
        SELECT precio_producto INTO NEW.precio_unitario
        FROM producto WHERE id_producto = NEW.producto_id;
    END IF;
    NEW.subtotal_detallepedido := NEW.cantidad_detallepedido * NEW.precio_unitario;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_subtotal
BEFORE INSERT OR UPDATE ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION fn_set_subtotal();

--
-- Trigger: total del pedido automático (AFTER ... FOR EACH STATEMENT)
--
CREATE OR REPLACE FUNCTION fn_recalcular_total()
RETURNS TRIGGER AS $$
BEGIN
    -- Recalcula el total de cada pedido afectado
    UPDATE pedido p
    SET total_pedido = calcular_total_pedido(p.id_pedido)
    WHERE p.id_pedido IN (SELECT pedido_id FROM afectados);

```

```

    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
-- Un trigger por evento: la transition table no admite declarar varios eventos en un mismo CREATE
-- TRIGGER
CREATE TRIGGER trg_total_ins
AFTER INSERT ON detalle_pedido
REFERENCING NEW TABLE AS afectados
FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
CREATE TRIGGER trg_total_upd
AFTER UPDATE ON detalle_pedido
REFERENCING NEW TABLE AS afectados
FOR EACH STATEMENT EXECUTE FUNCTION fn_recalcular_total();
--
-- Trigger genérico: soft delete (DELETE físico -> UPDATE eliminado = TRUE)
--
CREATE OR REPLACE FUNCTION fn_soft_delete()
RETURNS TRIGGER AS $$
DECLARE
    v_pk_column TEXT;
    v_pk_value BIGINT;
BEGIN
    v_pk_column := CASE TG_TABLE_NAME
        WHEN 'categoria' THEN 'id_categoria'
        WHEN 'producto' THEN 'id_producto'
        WHEN 'usuario' THEN 'id_usuario'
        WHEN 'pedido' THEN 'id_pedido'
        WHEN 'detalle_pedido' THEN 'id_detallepedido'
        ELSE NULL
    END;
    IF v_pk_column IS NULL THEN
        RAISE EXCEPTION 'fn_soft_delete: tabla % no soportada', TG_TABLE_NAME;
    END IF;
    v_pk_value := (to_jsonb(OLD) ->> v_pk_column)::BIGINT;
    EXECUTE format(
        'UPDATE %I SET eliminado = TRUE WHERE %I = $1 AND eliminado = FALSE',
        TG_TABLE_NAME, v_pk_column
    ) USING v_pk_value;
    RETURN NULL;
END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER trg_soft_delete_categoria
BEFORE DELETE ON categoria
FOR EACH ROW EXECUTE FUNCTION fn_soft_delete();
CREATE TRIGGER trg_soft_delete_producto
BEFORE DELETE ON producto
FOR EACH ROW EXECUTE FUNCTION fn_soft_delete();
CREATE TRIGGER trg_soft_delete_usuario
BEFORE DELETE ON usuario
FOR EACH ROW EXECUTE FUNCTION fn_soft_delete();
CREATE TRIGGER trg_soft_delete_pedido
BEFORE DELETE ON pedido

```

```

FOR EACH ROW EXECUTE FUNCTION fn_soft_delete();
CREATE TRIGGER trg_soft_delete_detalle_pedido
BEFORE DELETE ON detalle_pedido
FOR EACH ROW EXECUTE FUNCTION fn_soft_delete();
--
-- Procedimiento transaccional: alta de pedido + detalles
--
CREATE OR REPLACE PROCEDURE sp_crear_pedido(
    p_usuario_id BIGINT,
    p_forma_pago forma_pago,
    p_items JSONB -- [{"producto_id":1,"cantidad":2}, ...]
) AS $$
DECLARE
    v_pedido_id BIGINT;
    v_item JSONB;
    v_producto_id BIGINT;
    v_cantidad INTEGER;
    v_stock INTEGER;
    v_disponible BOOLEAN;
BEGIN
    -- El usuario debe existir y no estar eliminado
    IF NOT EXISTS (SELECT 1 FROM usuario
        WHERE id_usuario = p_usuario_id AND eliminado = FALSE) THEN -- corregido: id ->
id_usuario
        RAISE EXCEPTION 'Usuario % inexistente o eliminado', p_usuario_id;
    END IF;
    INSERT INTO pedido(usuario_id, forma_pago)
    VALUES (p_usuario_id, p_forma_pago)
    RETURNING id_pedido INTO v_pedido_id; -- corregido: id -> id_pedido
    FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
        v_producto_id := (v_item->>'producto_id')::BIGINT;
        v_cantidad := (v_item->>'cantidad')::INTEGER;
        -- Bloquea la fila del producto para evitar sobreventa concurrente
        SELECT stock_producto, disponible INTO v_stock, v_disponible
        FROM producto WHERE id_producto = v_producto_id AND eliminado = FALSE
        FOR UPDATE;
        IF NOT FOUND THEN
            RAISE EXCEPTION 'Producto % inexistente o eliminado', v_producto_id;
        END IF;
        IF NOT v_disponible THEN
            RAISE EXCEPTION 'Producto % no disponible', v_producto_id;
        END IF;
        IF v_stock < v_cantidad THEN
            RAISE EXCEPTION 'Stock insuficiente (producto %): hay %, se piden %',
                v_producto_id, v_stock, v_cantidad;
        END IF;
        INSERT INTO detalle_pedido(pedido_id, producto_id, cantidad_detallepedido)
        VALUES (v_pedido_id, v_producto_id, v_cantidad);
        -- Descuenta stock dentro de la misma transacción
        UPDATE producto SET stock_producto = stock_producto - v_cantidad
        WHERE id_producto = v_producto_id;
    END LOOP;

```

```

-- Si alguna inserción falla, toda la transacción se revierte (rollback).
END;
$$ LANGUAGE plpgsql;

```

6.2. Procedimiento **sp_crear_pedido**

Encapsula la creación del pedido en un bloque transaccional. Utiliza **SELECT ... FOR UPDATE** para bloquear el registro del producto y evitar sobreventas por solicitudes concurrentes.

```

CREATE OR REPLACE PROCEDURE sp_crear_pedido(
  p_usuario_id BIGINT,
  p_forma_pago forma_pago,
  p_items JSONB -- [{"producto_id":1,"cantidad":2}, ...]
) AS $$
DECLARE
  v_pedido_id BIGINT;
  v_item JSONB;
  v_producto_id BIGINT;
  v_cantidad INTEGER;
  v_stock INTEGER;
  v_disponible BOOLEAN;
BEGIN
  -- El usuario debe existir y no estar eliminado
  IF NOT EXISTS (SELECT 1 FROM usuario
    WHERE id_usuario = p_usuario_id AND eliminado = FALSE) THEN -- corregido: id ->
id_usuario
    RAISE EXCEPTION 'Usuario % inexistente o eliminado', p_usuario_id;
  END IF;
  INSERT INTO pedido(usuario_id, forma_pago)
  VALUES (p_usuario_id, p_forma_pago)
  RETURNING id_pedido INTO v_pedido_id; -- corregido: id -> id_pedido
  FOR v_item IN SELECT * FROM jsonb_array_elements(p_items) LOOP
    v_producto_id := (v_item->>'producto_id')::BIGINT;
    v_cantidad := (v_item->>'cantidad')::INTEGER;
    -- Bloquea la fila del producto para evitar sobreventa concurrente
    SELECT stock_producto, disponible INTO v_stock, v_disponible
    FROM producto WHERE id_producto = v_producto_id AND eliminado = FALSE
    FOR UPDATE;
    IF NOT FOUND THEN
      RAISE EXCEPTION 'Producto % inexistente o eliminado', v_producto_id;
    END IF;
    IF NOT v_disponible THEN
      RAISE EXCEPTION 'Producto % no disponible', v_producto_id;
    END IF;
    IF v_stock < v_cantidad THEN
      RAISE EXCEPTION 'Stock insuficiente (producto %): hay %, se piden %',
        v_producto_id, v_stock, v_cantidad;
    END IF;
    INSERT INTO detalle_pedido(pedido_id, producto_id, cantidad_detallepedido)
    VALUES (v_pedido_id, v_producto_id, v_cantidad);
    -- Descuenta stock dentro de la misma transacción

```

```

UPDATE producto SET stock_producto = stock_producto - v_cantidad
WHERE id_producto = v_producto_id;
END LOOP;
-- Si alguna inserción falla, toda la transacción se revierte (rollback).
END;
$$ LANGUAGE plpgsql;

```

7. Optimización con Índices y EXPLAIN ANALYZE

Probamos el rendimiento buscando un producto activo por nombre sobre un volumen masivo de datos:

SQL

- EXPLAIN ANALYZE
- SELECT * FROM producto WHERE nombre_producto = 'Hamburguesa Doble' AND eliminado = FALSE;

- Sin Índice Parcial: Seq Scan -> Tiempo de ejecución: 38.89 ms
- Con Índice Parcial (idx_producto_nombre_vigente): Index Scan -> Tiempo de ejecución: 0.061 ms (Reducción del 99.8% en tiempo de lectura).

8. Pruebas de Transacciones y Concurrency (transacciones.sql)

8.1. Atomicidad y Rollback

Si intentamos ejecutar `sp_crear_pedido` pasando un producto que no existe, la excepción detiene la ejecución. Se probó que **no queda generado ni el pedido ni los detalles previos**, garantizando el principio de todo o nada.

8.2. Control de Sobreventa con FOR UPDATE abiertas en paralelo:

SESIÓN 1 (Operador A)

SQL
BEGIN;

SQL
SELECT ... WHERE id_producto = 6 FOR UPDATE;
-- (Bloquea la fila)

SQL
CALL sp_crear_pedido(2, 'EFECTIVO', '{"producto_id": 6, "cantidad": 1}');

SQL
COMMIT;
-- (Stock pasa a 0)

SESIÓN 2 (Operador B)

SQL
BEGIN;

SQL
CALL sp_crear_pedido(3, 'EFECTIVO', '{"producto_id": 6, "cantidad": 1}');
-- (En espera por bloqueo)

Plaintext
-- (Despierta y lee stock_producto = 0)
-- ERROR: Stock insuficiente.

SQL
ROLLBACK;

9. Resolución de Historias de Usuario (queries.sql)

- **HU 1.1 — Alta de Categoría:**
- SQL
 - INSERT INTO categoria (nombre_categoria, descripcion_categoria) VALUES ('Empanadas', 'Variedad de empanadas');
-
-
- **HU 2.1 — Alta de Producto:**
- SQL
 - INSERT INTO producto (nombre_producto, precio_producto, stock_producto, categoria_id) VALUES ('Hamburguesa Doble', 4500.00, 20, 1);
-
-
- **HU 3.1 — Registro de Usuario:**
- SQL
 - INSERT INTO usuario (nombre_usuario, apellido_usuario, mail_usuario, contrasena_usuario) VALUES ('Niko', 'Pérez', 'niko@estudiante.utn.edu.ar', 'pass123');
-
-
- **HU 4.1 — Alta de Pedido:**
- SQL
 - CALL sp_crear_pedido(2, 'EFECTIVO', '{"producto_id": 2, "cantidad": 2}');

-
-
- **HU 4.2 — Consulta Analítica de Gastos por Cliente:**
- SQL
 - SELECT u.id_usuario, u.nombre_usuario || ' ' || u.apellido_usuario AS cliente, SUM(p.total_pedido) AS total_gastado
 - FROM usuario u JOIN pedido p ON u.id_usuario = p.usuario_id WHERE p.eliminado = FALSE
 - GROUP BY u.id_usuario, u.nombre_usuario, u.apellido_usuario ORDER BY total_gastado DESC;
-
-

10. Dificultades y Soluciones

1. **Alineación con el Esquema Oficial:**
 - *Problema:* El borrador inicial tenía nombres de columna genéricos (**id**, **nombre**).
 - *Solución:* Refactorizamos todo el código para respetar la nomenclatura prefijada por la cátedra (**id_usuario**, **nombre_categoria**, etc.).
2. **Evitar Recursión en Triggers:**
 - *Problema:* Actualizar el **total_pedido** desde un trigger de fila generaba bloqueos de tabla mutante.
 - *Solución:* Implementamos triggers **AFTER FOR EACH STATEMENT** usando tablas de transición (**REFERENCING NEW TABLE AS afectados**).

11. Conclusiones y Bibliografía

Logramos implementar una base de datos relacional robusta e íntegra para **Food Store**. La lógica vive en PostgreSQL mediante restricciones declarativas, funciones, triggers y transacciones aisladas, dejando todo optimizado para producción.

Bibliografía

1. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw-Hill.
2. PostgreSQL 16 Documentation. (2026). *Explicit Locking & PL/pgSQL*. PostgreSQL Global Development Group.